DS	**Session**	Queue	**Question Information**	Level 1 Challenge 58

Question description

Selvan is very interested in surfing the contents from google. He searches for various coding test on Google.

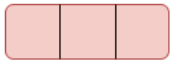
One day he searched about online coding competitions, in the retrieval links, he received many links for coding competition. he chooses first link from the goole suggestion list.

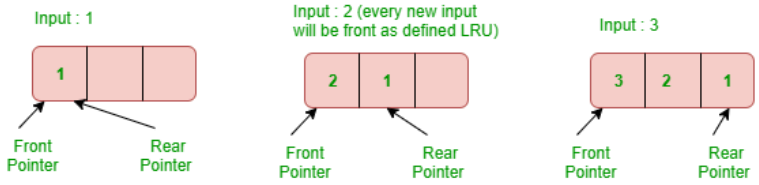
first question for the coding competition is LRU cache implementation using queue concepts.

Function Description

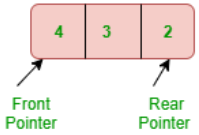
1. **Queue** which is implemented using a doubly linked list. The maximum size of the queue will be equal to the total number of frames available (cache size). The most recently used pages will be near front end and least recently pages will be near the rear end.
2. **A Hash** with page number as key and address of the corresponding queue node as value.

Given 3 page frames, so we take size of Queue is 3. Initially, Queue is empty.

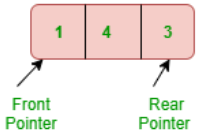




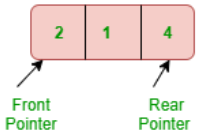
Input : 4 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



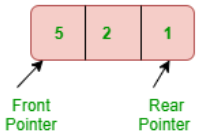
Input : 1 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



Input : 2 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



Input : 5 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



Constraints

For this experiment, cache can hold 4 pages.

Let 10 different pages can be requested (pages to be referenced are numbered from 0 to 9).

Input Format:

First line represents n and m, where n is the page number with in the range (0-9) & m is cache size (must be 4 for this problem).

Next line represents the reference pages.

Output Format:

Single line represents the cache frames after the above referenced pages.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
6 4
1 2 3 1 4 5
```

EXPECTED OUTPUT

```
5 4 1 3
```

Test Case 2

INPUT (STDIN)

```
10 4
3 1 4 5 9 8 7 6 1 2
```

EXPECTED OUTPUT

```
2 1 6 7
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
QNode* newQNode(unsigned pageNumber)
```

Test Case 2

KEYWORD

```
Queue* createQueue(int numberOfFrames)
```

Test Case 3

KEYWORD

```
typedef struct Queue
```

Test Case 4

KEYWORD

```
typedef struct QNode
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
1
```

Test Case 2

TOKEN COUNT

```
750
```

Test Case 3

NLOC

```
120
```

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct QNode {
    unsigned pageNumber;
    struct QNode *prev, *next;
} QNode;

typedef struct Queue {
    int count;
    int numberOfFrames;
    QNode *front, *rear;
} Queue;

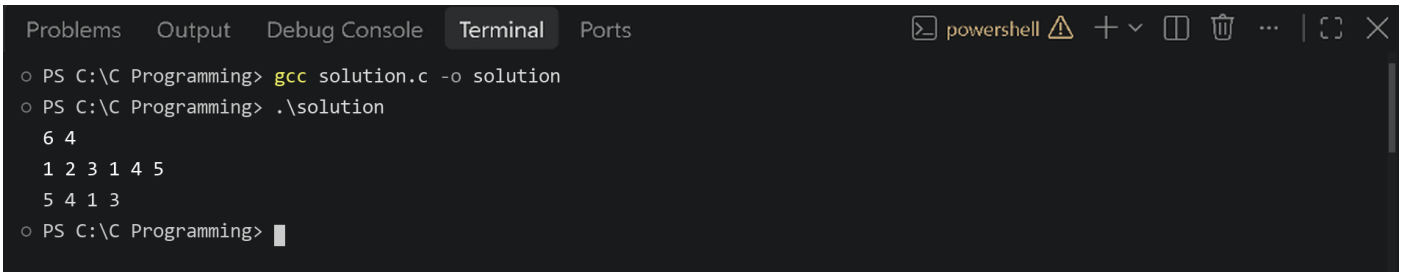
QNode* newQNode(unsigned pageNumber) {
    QNode* temp = (QNode*)malloc(sizeof(QNode));
    temp->pageNumber = pageNumber;
    temp->prev = temp->next = NULL;
    return temp;
}

Queue* createQueue(int numberOfFrames) {
    Queue* q = (Queue*)malloc(sizeof(Queue));
    q->count = 0;
    q->numberOfFrames = numberOfFrames;
    q->front = q->rear = NULL;
    return q;
}

void ReferencePage(Queue* q, unsigned pageNumber) {
    QNode* cur = q->front;
    while (cur) {
        if (cur->pageNumber == pageNumber) {
            if (cur != q->front) {
                if (cur == q->rear) {
                    q->rear = cur->prev;
                    q->rear->next = NULL;
                } else {
                    cur->prev->next = cur->next;
                    cur->next->prev = cur->prev;
                }
                cur->next = q->front;
                cur->prev = NULL;
                q->front->prev = cur;
                q->front = cur;
            }
            return;
        }
        cur = cur->next;
    }
    if (q->count == q->numberOfFrames) {
        QNode* del = q->rear;
        q->rear = q->rear->prev;
        if (q->rear) q->rear->next = NULL;
        else q->front = NULL;
        free(del);
        q->count--;
    }
    QNode* node = newQNode(pageNumber);
    node->next = q->front;
    if (q->front) q->front->prev = node;
    q->front = node;
    if (!q->rear) q->rear = node;
    q->count++;
}

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;
    Queue* q = createQueue(m);
    for (int i = 0; i < n; i++) {
        int p; scanf("%d", &p);
        ReferencePage(q, p);
    }
    QNode* cur = q->front;
    while (cur) {
        printf("%u%s", cur->pageNumber, cur->next ? " " : "");
        cur = cur->next;
    }
    printf("\n");
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
6 4
1 2 3 1 4 5
5 4 1 3
PS C:\C Programming>